Практическая работа №4

Тема:

Работа со связанными справочниками и расширенная настройка интерфейса (UI Customization)

Цель: создать справочник «Контрагенты», связать его с основной сущностью документа и выполнить стилизацию интерфейса под стандарты платформы.

Краткие теоретические сведения

Связи между сущностями (@ManyToOne). В реляционных базах данных для связи документа со справочником используется отношение «многие-к-одному». Например, множество документов могут ссылаться на одного и того же контрагента. В JPA это реализуется через аннотацию @ManyToOne и создание внешнего ключа (JoinColumn) в таблице документов.

ComboBox - основной компонент Vaadin для выбора значения из справочника. Он поддерживает фильтрацию «на лету» (Lazy Filtering), что важно, если в справочнике тысячи записей.

Lumo Theme и CSS - Vaadin использует стандартную тему Lumo. Для изменения внешнего вида (цветов, отступов, шрифтов) под требования корпоративного стиля Атом платформы используются CSS-переменные и аннотация @CssImport.

ThemeVariants - встроенные в компоненты Vaadin варианты оформления (например, LUMO_SMALL для компактности или LUMO_PRIMARY для акцентных кнопок).

Задание:

Часть 1. Создайте сущность справочника

Создайте класс Counterparty (Контрагент) с полями: id, name (краткое наименование), inn (ИНН), address.

Настройте соответствующий репозиторий и сервис.

В классе Document добавьте поле:

@ManyToOne

@JoinColumn(name = "counterparty_id")

private Counterparty counterparty;

Часть 2. Реализуйте интерфейс управления справочниками

Создайте отдельное представление (View) для ведения списка контрагентов. Реализуйте стандартный набор: таблица (Grid) и простая форма для добавления новых записей (по аналогии с ПР №2 и №3). Убедитесь, что справочник корректно сохраняет данные в БД.

Часть 3. Интегрируйте справочник в форму документа

В основной форме создания документа замените текстовое поле ввода контрагента на компонент ComboBox<Counterparty>.

Настройте отображение данных в выпадающем списке (метод setItemLabelGenerator должен возвращать наименование и ИНН).

Свяжите ComboBox с объектом через Binder

Часть 4. Стилизация и кастомизация (UI Tuning)

1. Примените к кнопкам формы различные варианты оформления (ThemeVariant.LUMO_PRIMARY для сохранения, ThemeVariant.LUMO_TERTIARY для отмены).
2. С помощью CSS измените цвет фона заголовка таблицы (Grid).
3. Сделайте поля формы более компактными, применив тему Lumo.VALUE_SMALL на уровне всего приложения или отдельных компонентов.

Содержание отчета

1. Скриншот кода сущности Document с настроенной связью @ManyToOne.

2. Скриншот формы документа, где виден открытый выпадающий список (ComboBox) с данными из справочника контрагентов.
3. Скриншот фрагмента CSS-файла или Java-кода, отвечающий за изменение стиля компонентов.
4. Скриншот итогового интерфейса, демонстрирующий применение компактной темы или измененных цветов.

Контрольные вопросы:

1. В чем преимущество хранения ссылок на справочники через ID (Foreign Key) вместо хранения текстовых названий прямо в таблице документов?
2. Какая аннотация в Vaadin позволяет подключить внешний CSS-файл к конкретному View?
3. Как ограничить высоту выпадающего списка в ComboBox, если в справочнике слишком много записей?
4. Чем FetchType.LAZY отличается от FetchType.EAGER?
5. Что такое нормализация базы данных?
6. Назовите три основные цели нормализации.
7. Какие три аномалии устраняет нормализация?
8. Сколько нормальных форм существует? Перечислите их по порядку.
9. Какие виды связей существуют в JPA?
10. Зачем нужен атрибут mappedBy?
11. Опишите словами, как в базе данных реализуется связь Many-to-Many. Какие таблицы появляются?
12. Что произойдет, если в справочнике будет 1 000 000 записей, а ComboBox загрузит их все?

Список полезных источников:

1. Официальная документация Vaadin, «ComboBox» - https://vaadin.com/docs/latest/components/combo-box

2. Хабрахабр, «Односторонние и двусторонние отношения в Hibernate» - https://habr.com/ru/articles/542328/
3. Baeldung, «Introduction to Vaadin» - https://www.baeldung.com/vaadin